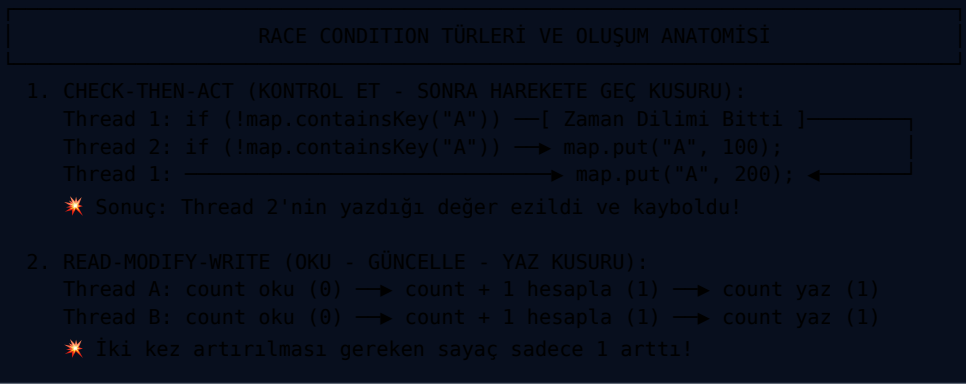
49

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Klasik Yarış Durumları (Race Conditions): Check-Then-Act ve Read-Modify-Write Kusurları

Classic Race Conditions: Check-Then-Act & Read-Modify-Write Flaws

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Check-then-act, banyoya girmeden önce kapıyı tıklatıp içeride kimse olmadığını görmek, tam içeri adım atacakken başka birinin yan kapıdan hızla banyoya girmesi gibidir. Sen 'Banyo boştu!' diyerek içeri dalarsın ve felaket yaşanır! Kontrol anı ile eylem anı arasında geçen mikrosaniyelik boşlukta evren değişmiştir. Read-modify-write ise aynı anda iki kişinin buzdolabında kalan son yumurtayı görüp ikisinin de tavaya kırmasıdır; sonunda yumurta yok olur!

🇬🇧 ENGLISH (ELI5)

Check-then-Act is knocking on a dressing room door, confirming it is empty, and pausing for one second before stepping in. In that one split-second, someone else slips inside. You burst in thinking it is still vacant, causing chaos. The state observed during the 'check' was invalidated before the 'act'. Read-Modify-Write is two cooks reading an inventory sheet showing 1 egg left, both taking that egg to bake a cake simultaneously, resulting in missing inventory!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kritik Bölge Zafiyetleri ve Atomik Çözümler: 1) Check-Then-Act: En yaygın örneği `Map.containsKey()` ardından `Map.put()` yapılması veya tembel yüklemedeki (Lazy Init) `if (instance == null)` kontrolüdür. Kontrol ile yazma arasında atomik bir bağ yoktur. Eşzamanlı ortamda durum başka bir thread tarafından değiştirilebilir. Çözüm: `ConcurrentMap.putIfAbsent()` veya `computeIfAbsent()` gibi donanımsal CAS tabanlı atomik primitifler kullanmaktır. 2) Read-Modify-Write: `count++` işlemi aslında 3 bytecode komutudur (`getfield`, `iadd`, `putfield`). İki thread aynı anda `getfield` yaparsa biri diğerinin artırmasını ezer. Çözüm: `AtomicInteger.incrementAndGet()` veya `synchronized` kilididir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Critical Section Vulnerabilities: 1) Check-Then-Act: Arises when a state inspection informs a subsequent mutation, but the observed state becomes stale before the mutation executes. Common in compound collections (`if (map.containsKey(k)) map.put(k, v)`). Resolution: Leverage compound atomic methods like `ConcurrentMap.putIfAbsent()` or `computeIfAbsent()`. 2) Read-Modify-Write: Manifests in expressions like `count++`, which compile down to three discrete bytecode instructions (`getfield`, `iadd`, `putfield`). When interleaved, concurrent threads read identical initial values, discarding increments. Resolution: Atomic CAS wrappers (`AtomicInteger.incrementAndGet()`) or mutual exclusion locks.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Check-then-Act Hatası ve Modern Atomik Çözümü
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;

public class RaceConditionDemo {
    private final ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();

    // ❌ HATALI: Check-then-act yarış durumu (Compound Operation)
    public void unsafePut(String key, String value) {
        if (!cache.containsKey(key)) { // Kontrol
            // ⚠ TAM BURADA başka bir thread araya girip key'i ekleyebilir!
            cache.put(key, value);     // Eylem
        }
    }

    // ✅ DOĞRU: Tek adımda atomik işlem (Atomic Compound)
    public void safePut(String key, String value) {
        cache.putIfAbsent(key, value); // JVM ve Donanım seviyesinde atomik!
    }

    void main() {
        var demo = new RaceConditionDemo();
        try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 1000; i++) {
                final int id = i;
                pool.submit(() -> demo.safePut("CONFIG_KEY", "VAL_" + id));
            }
        }
        System.out.println("Nihai güvenli değer: " + demo.cache.get("CONFIG_KEY"));
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ConcurrentHashMap kullanmak tek başına tüm yarış durumlarını ÇÖZMEZ! Eğer haritadan okuyup (get), bir hesaplama yapıp, sonra tekrar yazıyorsanız (put), iki metodun arası atomik değildir ve yarış durumu yaşanır. Her zaman compute(), computeIfAbsent() veya merge() kullanmalısınız!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Merely wrapping state in `ConcurrentHashMap` does NOT immunize against race conditions! If your code performs `map.get()`, calculates a new value, and issues `map.put()`, the interval between calls is unguarded. You must encapsulate compound state transitions within `compute()` or `merge()`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Birden fazla adımdan oluşan işlemlerde araya başka thread girebilir. Kontrol ve eylem mutlaka tek bir atomik operasyonda birleştirilmelidir.

🎯 Key Takeaways for the Interview

• Compound actions without isolation permit thread interleaving. Merge check-and-act steps into single atomic operations.